

MONOGRAFÍA

DESARROLLO DE APLICACIONES MÓVILES CON ANDROID STUDIO

Estudiante:

CHAMORRO ARIAS YEYSON ERICK

Docente:

BARRA ESPINOZA ANTHONY HELI

2026

ÍNDICE

1.	Introducción	3
2.	Introducción a Android Studio	4
3.	Fundamentos del Desarrollo en Android	6
4.	Programación en Java para Android	8
5.	Diseño de Interfaces Gráficas	11
6.	Ciclo de Vida de una Activity	14
7.	Estilos, Temas y Eventos	16
8.	Base de Datos en Android	18
9.	Consumo de APIs	21
10.	Manejo y Persistencia de Datos	24
11.	Publicación de Aplicaciones	27
12.	Conclusiones	29
13.	Bibliografía	30

. INTRODUCCIÓN

El desarrollo de aplicaciones móviles se ha convertido en una de las habilidades más demandadas en la industria tecnológica actual. Android, el sistema operativo móvil de Google, domina el mercado global con más del 70% de cuota, lo que hace que el conocimiento de su plataforma de desarrollo sea esencial para cualquier programador moderno.

La presente monografía tiene como objetivo reunir, organizar y explicar de manera comprensible los principales conceptos abordados durante el curso de Programación Móvil (IP_EMPRESA). A lo largo de este documento, el estudiante encontrará desde los fundamentos básicos de la instalación del entorno de desarrollo hasta temas avanzados como el consumo de APIs REST, manejo de bases de datos locales y la publicación de aplicaciones en Google Play Store.

Cada sección ha sido elaborada con un lenguaje accesible, acompañada de ejemplos prácticos y fragmentos de código que ilustran los conceptos descritos. Se parte de la premisa de que el aprendizaje efectivo requiere no solo la comprensión teórica, sino también la capacidad de aplicar dichos conocimientos en proyectos reales.

El recorrido que ofrece esta monografía es progresivo: primero se establecen las bases del entorno de desarrollo con Android Studio; luego se exploran los fundamentos de la arquitectura Android; posteriormente se profundiza en el lenguaje Java como motor de la lógica de aplicaciones; se examina el diseño de interfaces, el ciclo de vida de las actividades, los estilos y eventos, la gestión de datos y bases de datos, el consumo de servicios web y, finalmente, el proceso de publicación de una aplicación.

Es el propósito de este trabajo demostrar una comprensión integral del ecosistema Android, evidenciar el aprendizaje obtenido durante el curso y servir como material de consulta y referencia para proyectos futuros.

1. INTRODUCCIÓN A ANDROID STUDIO

1.1 ¿Qué es Android Studio?

Android Studio es el Entorno de Desarrollo Integrado (IDE) oficial para la creación de aplicaciones Android. Fue lanzado por Google en 2014 y está basado en IntelliJ IDEA de JetBrains. Proporciona todas las herramientas necesarias para diseñar, codificar, depurar y publicar aplicaciones Android en un solo lugar.

1.2 Instalación y Configuración

Para instalar Android Studio se deben seguir estos pasos:

- Descargar el instalador desde developer.android.com/studio.
- Ejecutar el instalador y seleccionar los componentes: Android SDK, Android Virtual Device (AVD) y las herramientas de compilación.
- Configurar la variable de entorno `ANDROID_HOME` apuntando al directorio del SDK.
- Verificar la instalación ejecutando el IDE y creando un proyecto de prueba (Hello World).

Los requisitos mínimos del sistema son: 8 GB de RAM (16 GB recomendado), 8 GB de espacio en disco, resolución de pantalla 1280×800 y sistema operativo Windows 8/10/11, macOS 10.14+ o Linux.

1.3 Configuración del Emulador de Android (AVD)

El Android Virtual Device Manager (AVD) permite ejecutar y probar aplicaciones sin necesidad de un dispositivo físico. Para configurarlo:

- Abrir Android Studio → Tools → Device Manager.
- Hacer clic en 'Create Virtual Device'.
- Seleccionar el tipo de dispositivo (teléfono, tablet, Wear OS, etc.).
- Elegir una imagen del sistema (por ejemplo Android 14.0 – API 34).
- Configurar la RAM, el almacenamiento y la resolución del emulador.

Consejo práctico

Si el emulador es lento, se recomienda habilitar la aceleración por hardware (recomendado el celular). Esto puede acelerar el emulador hasta 10 veces.

1.4 Uso de Gradle en Proyectos Android

Gradle es el sistema de automatización de construcción (build system) utilizado en Android Studio. Se encarga de compilar el código fuente, empaquetar recursos, gestionar dependencias y generar el archivo APK o AAB final.

Cada proyecto Android tiene dos archivos Gradle principales:

- build.gradle (Project level): configuración global del proyecto, repositorios Maven y Google.
- build.gradle (App level): versión del SDK, dependencias específicas del módulo, tipo de firma.

Ejemplo de bloque de dependencias en build.gradle:

```
dependencies {  
    implementation("androidx.recyclerview:recyclerview:1.3.2")  
    implementation("androidx.cardview:cardview:1.0.0")  
    implementation("com.google.android.material:material:1.12.0")  
    implementation("androidx.fragment:fragment:1.6.2")  
    implementation("com.github.PhilJay:MPAndroidChart:v3.1.0")  
    implementation(dependencyNotation = "org.apache.poi:poi-ooxml:3.17") {  
        exclude(group = "org.apache.xmlbeans", module = "xmlbeans")  
    }  
    implementation("androidx.biometric:biometric:1.1.0")  
}
```

Cuando se sincroniza el proyecto (Sync Now), Gradle descarga automáticamente las bibliotecas declaradas desde los repositorios configurados (Google Maven, Maven Central, JCenter).

2. FUNDAMENTOS DEL DESARROLLO EN ANDROID

2.1 Arquitectura de una Aplicación Android

Una aplicación Android está estructurada en múltiples capas que separan responsabilidades y facilitan el mantenimiento del código. La arquitectura recomendada por Google (basada en los principios de Clean Architecture) divide la aplicación en tres capas principales:

CAPA	DESCRIPCIÓN
UI / Presentación	Activities, Fragments, Views. Muestra datos al usuario e interpreta sus acciones.
Dominio / Negocio	ViewModels, UseCases. Contiene la lógica de negocio de la aplicación.
Datos	Repositories, Room, Retrofit. Gestiona el acceso a datos locales y remotos.

2.2 Componentes Principales de una App Android

Android define cuatro tipos de componentes fundamentales que conforman el bloque de construcción de cualquier aplicación:

Activity: Es el componente que representa una pantalla con interfaz de usuario. Cada vez que el usuario navega a una nueva pantalla, se crea o retoma una Activity. Por ejemplo, en una aplicación de correo electrónico, la pantalla de bandeja de entrada es una Activity, y la pantalla de redacción de un correo es otra.

Fragment: Un Fragment es una porción reutilizable de interfaz de usuario que vive dentro de una Activity. Permite construir interfaces adaptables para teléfonos y tabletas. Por ejemplo, en una tablet se puede mostrar una lista y el detalle simultáneamente usando dos Fragments dentro de una sola Activity.

View y Layout: Las Views son los elementos visuales individuales (botones, textos, imágenes), mientras que los Layouts son contenedores que organizan visualmente esas Views en la pantalla.

Service: Componente que ejecuta tareas en segundo plano sin interfaz de usuario (por ejemplo, reproducir música o descargar archivos).

BroadcastReceiver: Permite a la aplicación responder a eventos del sistema, como la recepción de un SMS o el cambio de conectividad de red.

ContentProvider: Gestiona el acceso compartido a datos entre aplicaciones (por ejemplo, acceder a los contactos del dispositivo).

3. PROGRAMACIÓN EN JAVA PARA ANDROID

Java es el lenguaje de programación principal utilizado en Android durante muchos años. Aunque Kotlin es ahora el lenguaje oficial recomendado por Google, Java sigue siendo ampliamente utilizado y completamente compatible con el SDK de Android.

3.1 Tipos de Datos y Operadores

Java es un lenguaje fuertemente tipado, lo que significa que cada variable debe declararse con un tipo de dato específico. Los tipos primitivos más usados en Android son:

```
// Tipos primitivos
no usages
int    edad    = 25;           // Entero (4 bytes)
no usages
double precio = 19.99;        // Decimal doble precisión
no usages
boolean activo = true;        // Verdadero / Falso
no usages
char    inicial = 'A';         // Carácter Unicode
// Tipos de referencia (objetos)
no usages
String  nombre  = "Android";   // Cadena de texto
no usages
int[]   numeros = {1, 2, 3};   // Arreglo de enteros
```

Los operadores en Java se clasifican en: aritméticos (+, -, *, /, %), relacionales (==, !=, <, >, <=, >=), lógicos (&&, ||, !), de asignación (=, +=, -=, *=) y el operador ternario (condición ? valorSi : valorNo).

3.2 Estructuras de Control

Las estructuras de control permiten tomar decisiones y repetir operaciones. Las más utilizadas en Android son:

```
// if-else: validar formulario en Android
String usuario = etUsuario.getText().toString();
if (usuario.isEmpty()) {
    Toast.makeText(this, "El usuario no puede estar vacío",
        Toast.LENGTH_SHORT).show();
} else {
    // Proceder con el inicio de sesión
    iniciarSesion(usuario);
}

// switch: navegar entre pantallas
switch (opcionMenu) {
    case 1: startActivity(new Intent(this, HomeActivity.class)); break;
    case 2: startActivity(new Intent(this, PerfilActivity.class)); break;
    default: Log.w("Menu", "Opción no reconocida"); break;
}

// for: llenar un ListView
for (int i = 0; i < listaProductos.size(); i++) {
    Log.d("Producto", listaProductos.get(i).getNombre());
}
```

3.3 Métodos y Funciones en Java

Los métodos son bloques de código reutilizables que realizan una tarea específica. Su estructura básica es: modificador de acceso + tipo de retorno + nombre + parámetros + cuerpo.

```
// Método que valida un correo electrónico
no usages
private boolean esCorreoValido(String correo) {
    return correo != null && correo.contains("@") && correo.contains(".");
}

// Método que calcula el IMC
no usages
public double calcularIMC(double peso, double altura) {
    if (altura <= 0) return 0;
    return peso / (altura * altura);
}

Método void (sin retorno) que muestra un Toast
```

En Android, los métodos privados (private) son usados para lógica interna de la Activity, mientras que los métodos públicos (public) pueden ser llamados desde otras clases. Es buena práctica mantener los métodos pequeños, con una única responsabilidad clara.

4. DISEÑO DE INTERFACES GRÁFICAS

El diseño de la interfaz de usuario (UI) es uno de los aspectos más importantes en el desarrollo de aplicaciones Android, pues determina la experiencia del usuario. Android utiliza archivos XML para definir la estructura visual de cada pantalla.

4.1 Introducción a Android UI y XML

Las interfaces en Android se definen mediante archivos XML ubicados en la carpeta res/layout/. Estos archivos describen la jerarquía de vistas: qué elementos hay, cómo se disponen y cuáles son sus propiedades visuales.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F1F8E9"
    android:orientation="vertical">

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
```

4.2 Tipos de Layouts

Un Layout es un contenedor invisible que organiza las Views en la pantalla. Android ofrece varios tipos:

LinearLayout: Organiza sus hijos en una única fila horizontal o columna vertical. Es simple y predecible. Su propiedad más importante es android:orientation (horizontal | vertical).

ConstraintLayout: El más potente y recomendado actualmente. Permite posicionar cualquier vista mediante restricciones (constraints) relativas a otros elementos o al padre. Elimina la necesidad de anidar múltiples layouts, lo que mejora el rendimiento.

RelativeLayout: Organiza los elementos relativamente entre sí (por ejemplo, 'colocar este botón debajo de ese TextView'). Ha sido reemplazado en gran medida por ConstraintLayout, pero sigue siendo útil para layouts simples.

Buena práctica

Se recomienda usar ConstraintLayout para interfaces complejas ya que evita el anidamiento profundo de layouts, lo que puede causar problemas de rendimiento al renderizar la pantalla.

4.3 Componentes Visuales Principales

TextView: Muestra texto en pantalla. Se puede personalizar con textSize (tamaño en sp), textColor (color), textStyle (bold, italic), gravity (alineación) y muchos otros atributos.

Button: Componente interactivo que el usuario puede presionar para ejecutar una acción. Puede tener texto, un ícono o ambos. Se le asigna un evento mediante setOnClickListener en Java o con el atributo android:onClick en XML.

ImageView: Muestra imágenes en la pantalla. Admite imágenes locales (en res/drawable), imágenes del dispositivo o imágenes cargadas desde Internet mediante bibliotecas como Glide o Picasso.

```
// En Java: obtener referencias a las vistas del XML
TextView tvNombre = findViewById(R.id.tvNombre);
Button btnGuardar = findViewById(R.id.btnGuardar);
ImageView ivFoto = findViewById(R.id.ivFoto);

// Modificar propiedades en tiempo de ejecución
tvNombre.setText("María García");
tvNombre.setTextColor(Color.parseColor("#2E7D32"));

btnGuardar.setOnClickListener(view -> {
    guardarDatos();
});
```

5. CICLO DE VIDA DE UNA ACTIVITY

El ciclo de vida de una Activity es uno de los conceptos más importantes en Android. Comprender cuándo se ejecuta cada método del ciclo de vida es esencial para crear aplicaciones que no pierdan datos, no consuman recursos innecesariamente y no se cuelguen ante rotaciones de pantalla u otras interrupciones del sistema.

5.1 Estados del Ciclo de Vida

Android define seis métodos de callback que marcan los estados de una Activity:

MÉTODO	DESCRIPCIÓN Y USO TÍPICO
<code>onCreate()</code>	Se llama cuando la Activity se crea por primera vez. Aquí se inicializa la UI (<code>setContentView()</code>), se obtienen referencias a las vistas (<code>findViewById()</code>) y se restaura el estado guardado.
<code>onStart()</code>	La Activity se vuelve visible para el usuario. Ideal para registrar receptores de eventos o iniciar animaciones.
<code>onResume()</code>	La Activity está en primer plano e interactúa con el usuario. Es el estado activo. Aquí se reanudan tareas como la reproducción de video.
<code>onPause()</code>	La Activity pierde el foco (p.ej., llega una llamada). Se detienen procesos que no deben continuar (animaciones, GPS). Los datos críticos deben guardarse aquí.
<code>onStop()</code>	La Activity ya no es visible. Se liberan recursos pesados como conexiones de base de datos y se guardan datos persistentes.
<code>onDestroy()</code>	La Activity es destruida definitivamente. Se liberan todos los recursos restantes para evitar fugas de memoria.

Un ejemplo práctico: en una aplicación de mapas, se inicializa el mapa en `onCreate`, se solicita permisos de ubicación en `onResume`, se detiene la actualización de ubicación en `onPause` (para no agotar la batería) y se libera el objeto de mapa en `onDestroy`.

```
@Override
public void onCreate() {
    super.onCreate();
    // Aplica modo oscuro / claro global al iniciar
    ThemeManager.get(this).inicializar();
}
```

6. ESTILOS, TEMAS Y EVENTOS

6.1 Uso de Material Design

Material Design es el sistema de diseño visual creado por Google que establece principios de diseño coherentes, modernos y elegantes para aplicaciones Android. Proporciona componentes preconstruidos como `MaterialButton`, `TextInputLayout`, `BottomNavigationView`, `FloatingActionButton`, entre muchos otros.

Para usar Material Design en un proyecto, se debe agregar la dependencia en `build.gradle` y asignar un tema Material en `AndroidManifest.xml` o en `res/values/themes.xml`.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  ⚡ <style name="Theme.Ecolim" parent="Theme.MaterialComponents.DayNight.NoActionBar"
    <!-- Colores primarios -->
    <item name="colorPrimary">@color/verde_primary</item>
    <item name="colorPrimaryVariant">@color/verde_dark</item>
    <item name="colorOnPrimary">#FFFFFF</item>

    <!-- Colores secundarios -->
    <item name="colorSecondary">@color/verde_secondary</item>
    <item name="colorSecondaryVariant">@color/verde_dark</item>
    <item name="colorOnSecondary">#FFFFFF</item>
```

6.2 Personalización de Colores y Tipografías

Los colores se definen en `res/values/colors.xml` y se referencian en el código con `@color/nombre` o en Java con `R.color.nombre`. Para las tipografías, Android soporta fuentes personalizadas colocadas en `res/font/` y referenciadas mediante el atributo `android:fontFamily`.

6.3 Manejo de Eventos en la Interfaz

Los eventos permiten que la aplicación reaccione a las interacciones del usuario. Los más comunes son:

```
// onClick: toque simple sobre un botón
btnLogin.setOnClickListener(v -> {
    validarCredenciales();
});

// onLongClick: mantener presionado (para opciones adicionales)
ivFoto.setOnLongClickListener(v -> {
    mostrarMenuContextual();
    return true; // Indica que el evento fue consumido
});

// Evento de cambio de texto en un EditText
etBuscar.addTextChangedListener(new TextWatcher() {
    @Override
    public void onTextChanged(CharSequence s, int start, int before, int count) {
        filtrarResultados(s.toString());
    }
    // También: beforeTextChanged y afterTextChanged
});
```

6.4 Toast, Snackbar y AlertDialog

Android ofrece tres componentes principales para mostrar mensajes y confirmaciones al usuario:

Toast: Mensaje flotante breve que aparece y desaparece automáticamente. Ideal para confirmaciones simples como 'Guardado exitosamente'.

Snackbar: Similar al Toast pero aparece en la parte inferior de la pantalla y puede incluir una acción (por ejemplo: 'DESHACER'). Es la alternativa moderna y recomendada por Material Design.

AlertDialog: Cuadro de diálogo modal que requiere una respuesta del usuario antes de continuar. Ideal para confirmar acciones destructivas como 'Eliminar registro'.

```
// Toast
Toast.makeText(this, "Datos guardados", Toast.LENGTH_SHORT).show();

// Snackbar con acción
Snackbar.make(binding.root, "Elemento eliminado", Snackbar.LENGTH_LONG)
    .setAction("DESHACER", v -> restaurarElemento())
    .show();

// AlertDialog
new AlertDialog.Builder(this)
```

```
.setTitle("Confirmar eliminación")
.setMessage("¿Estás seguro de que deseas eliminar este registro?")
.setPositiveButton("Eliminar", (dialog, which) -> eliminarRegistro())
.setNegativeButton("Cancelar", null)
.show();
```

7. BASE DE DATOS EN ANDROID

7.1 Comunicación entre Activities y Fragments

Para pasar información de una Activity a otra se utilizan los Intents con extras. Un Intent es un objeto que describe una operación a realizar (abrir una pantalla, compartir contenido, etc.) y puede transportar datos adicionales en formato clave-valor.

```
// Activity de origen: enviar datos
Intent intent = new Intent(MainActivity.this, DetalleActivity.class);
intent.putExtra("producto_id", 42);
intent.putExtra("producto_nombre", "Laptop ASUS");
startActivity(intent);

// Activity de destino: recibir datos
int id = getIntent().getIntExtra("producto_id", -1);
String nombre = getIntent().getStringExtra("producto_nombre");
```

Para comunicar un Fragment con su Activity o con otro Fragment, se utilizan interfaces de callback, ViewModel compartido o el método getArguments() con Bundle.

7.2 Uso de SQLite

SQLite es el motor de base de datos relacional embebido en Android. Permite almacenar datos estructurados de forma local en el dispositivo, sin necesidad de conexión a internet. Cada aplicación tiene su propia base de datos SQLite aislada.

Para trabajar con SQLite en Android se extiende la clase SQLiteOpenHelper, que gestiona la creación y actualización de la base de datos.

```
public class ThemeManager {  
    1 usage  
    private static final String PREFS = "ecolim_prefs";  
    2 usages  
    private static final String KEY_DARK = "modo_oscuro";  
    2 usages  
    private static final String KEY_CONTRAST = "alto_contraste";  
    2 usages  
    private static final String KEY_COLOR = "color_tema";  
    2 usages  
    private static final String KEY_FONT = "font_size";  
}
```

7.3 Operaciones CRUD

CRUD son las cuatro operaciones básicas sobre cualquier base de datos: Create (insertar), Read (leer), Update (actualizar) y Delete (eliminar).

```
private void insertarUsuarioAdmin(SQLiteDatabase db) {  
    ContentValues v = new ContentValues();  
    v.put(U_NOMBRE, "Administrador");  
    v.put(U_APELLIDO, "ECOLIM");  
    v.put(U_DNI, "000000000");  
    v.put(U_CARGO, "Supervisor");  
    v.put(U_ROL, "ADMIN"); // ← no  
    v.put(U_EMAIL, "admin@ecolim.com");  
    v.put(U_PASSWORD, "admin123");  
    v.put(U_TELEFONO, "999000000");  
    v.put(U_FECHA_REGISTRO, "2024-01-01");  
    v.put(U_ACTIVO, 1);  
    db.insert(T_USUARIOS, nullColumnHack: null, v);  
}
```

8. CONSUMO DE APIs

Las aplicaciones modernas raramente funcionan de forma completamente offline. La mayoría consumen servicios web (APIs) para obtener datos actualizados, autenticar usuarios, procesar pagos, etc. En Android, esto se realiza mediante peticiones HTTP a APIs REST.

8.1 Introducción a HTTP y APIs REST

HTTP (HyperText Transfer Protocol) es el protocolo de comunicación fundamental de la web. Una API REST (Representational State Transfer) es una interfaz que permite a dos aplicaciones comunicarse a través de HTTP, intercambiando datos generalmente en formato JSON.

Los métodos HTTP más utilizados en Android son:

- GET: Solicita datos del servidor (p.ej., obtener lista de productos).
- POST: Envía datos al servidor para crear un nuevo recurso (p.ej., registrar usuario).
- PUT/PATCH: Actualiza un recurso existente en el servidor.
- DELETE: Elimina un recurso del servidor.

8.2 Uso de Retrofit y OkHttp

Retrofit es la biblioteca de networking más popular para Android, desarrollada por Square. Simplifica enormemente el consumo de APIs REST convirtiendo automáticamente las respuestas JSON en objetos Java.

OkHttp es el cliente HTTP subyacente que Retrofit utiliza por defecto. También puede usarse directamente para casos donde se necesita mayor control sobre las peticiones.

```
public interface ApiService {  
    1 usage  
    @POST("login")  
    Call<LoginResponse> loginUser(@Body LoginRequest request);  
    no usages  
    @POST("register")  
    Call<LoginResponse> registerUser(@Body LoginRequest request);  
    no usages  
    @GET("users/2")  
    Call<UserProfileResponse> getUserProfile(  
        @Header("Authorization") String token
```

Permisos de Internet

Para que la aplicación pueda hacer peticiones de red, es obligatorio declarar el permiso en `AndroidManifest.xml`: `<uses-permission android:name="android.permission.INTERNET" />`

9. MANEJO Y PERSISTENCIA DE DATOS

9.1 Uso de JSON

JSON (JavaScript Object Notation) es el formato de intercambio de datos más utilizado en APIs REST. Es legible por humanos, ligero y fácilmente procesable en Java. Un objeto JSON se representa con llaves {} y los arreglos con corchetes [].

```
// Ejemplo de respuesta JSON de una API de productos
{
  "status": "ok",
  "total": 2,
  "productos": [
    { "id": 1, "nombre": "Laptop", "precio": 999.99 },
    { "id": 2, "nombre": "Teclado", "precio": 49.99 }
  ]
}
```

9.2 Serialización con Gson y Moshi

La serialización convierte objetos Java en JSON (y viceversa). Gson (de Google) y Moshi (de Square) son las bibliotecas más populares para esta tarea en Android. Gson es más simple de usar; Moshi es más eficiente y maneja mejor los valores null.

```
// GSON: convertir JSON a objeto Java
Gson gson = new Gson();
Producto p = gson.fromJson(jsonString, Producto.class);

// GSON: convertir objeto Java a JSON
String json = gson.toJson(p);

// Modelo con anotaciones Gson
public class Producto {
    @SerializedName("id") private int id;
    @SerializedName("nombre") private String nombre;
    @SerializedName("precio") private double precio;
    // getters y setters...
}
```

9.3 SharedPreferences

SharedPreferences es el mecanismo más simple para guardar datos pequeños de configuración en formato clave-valor (preferencias de usuario, token de sesión, configuraciones de la app). Los datos persisten aunque la aplicación sea cerrada.

```
// Guardar datos
SharedPreferences prefs = getSharedPreferences("mi_app", MODE_PRIVATE);
prefs.edit()
    .putString("token", "eyJhbGciOiJIUzI1NiJ9...")
    .putBoolean("logueado", true)
    .apply();

// Leer datos
String token = prefs.getString("token", "");
boolean logueado = prefs.getBoolean("logueado", false);
```

9.4 Room Database

Room es la biblioteca de persistencia de Android que proporciona una capa de abstracción sobre SQLite. Permite trabajar con bases de datos de forma más segura, con verificación en tiempo de compilación y soporte nativo para LiveData y Coroutines.

Room se basa en tres componentes principales: Entity (tabla de la BD), DAO (Data Access Object, define las consultas) y Database (clase principal que une todo).

```
@Entity(tableName = "usuarios")
public class Usuario {
    @PrimaryKey(autoGenerate = true)
    public int id;
    public String nombre;
    public String email;
}

@Dao
public interface UsuarioDao {
    @Insert
    void insertar(Usuario u);

    @Query("SELECT * FROM usuarios")
    LiveData<List<Usuario>> obtenerTodos();

    @Delete
    void eliminar(Usuario u);
}
```

9.5 LiveData y ViewModel

LiveData es un contenedor de datos observable que respeta el ciclo de vida de Android. Cuando los datos cambian, notifica automáticamente a los observadores activos (Activities o Fragments), eliminando la necesidad de actualizar manualmente la UI.

ViewModel es un componente que almacena y gestiona los datos relacionados con la UI de manera que sobreviven a los cambios de configuración (como rotaciones de pantalla). Separa la lógica de presentación de los componentes de UI.

10. PUBLICACIÓN DE APLICACIONES

10.1 Registro en Google Play Console

Google Play Console es la plataforma de Google donde los desarrolladores gestionan sus aplicaciones y las publican en Google Play Store. Para registrarse se requiere una cuenta de Google y el pago de una única tarifa de registro de \$25 USD.

Una vez registrado, el proceso de creación de una aplicación en Play Console incluye:

- Crear la ficha de la aplicación con nombre, descripción, capturas de pantalla e icono.
- Completar las declaraciones de política de privacidad y seguridad de datos.
- Definir el tipo de aplicación (gratuita, de pago, con compras integradas).
- Seleccionar los países y regiones de distribución.

10.2 Requisitos para Publicar una Aplicación

Para publicar una app en Google Play, se deben cumplir los siguientes requisitos técnicos y de contenido:

- Generar un APK firmado o un Android App Bundle (AAB) con una clave de firma (keystore).
- Definir la versión de la aplicación (versionCode y versionName en build.gradle).
- Cumplir con las políticas de contenido de Google Play (sin malware, sin contenido engañoso).
- Declarar todos los permisos utilizados y justificar su uso si son sensibles.
- Incluir política de privacidad si la app recopila datos personales.

Para generar el AAB firmado en Android Studio: Build → Generate Signed Bundle/APK → Android App Bundle → crear o seleccionar keystore → configurar alias y contraseña → Build.

```
// build.gradle (App): Configuración de firma para producción
android {
    defaultConfig {
        applicationId "com.empresa.miapp"
        versionCode 1
        versionName "1.0.0"
    }
    signingConfigs {
        release {
```

```
        storeFile      file("mi_keystore.jks")
        storePassword  "mi_contraseña"
        keyAlias       "mi_alias"
        keyPassword    "mi_clave"
    }
}
buildTypes {
    release {
        signingConfig signingConfigs.release
        minifyEnabled true
        shrinkResources true
    }
}
}
```

10.3 Seguridad y Permisos en Android

Android implementa un sistema de permisos para proteger la privacidad del usuario y los recursos del sistema. Los permisos se clasifican en:

Permisos normales: Se conceden automáticamente al instalar la app (INTERNET, VIBRATE). No requieren aprobación del usuario.

Permisos peligrosos: Requieren que el usuario los apruebe explícitamente en tiempo de ejecución (a partir de Android 6.0). Ejemplos: CAMERA, READ_CONTACTS, ACCESS_FINE_LOCATION, READ_EXTERNAL_STORAGE.

```
// Solicitar permiso de cámara en tiempo de ejecución
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
    != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
        new String[]{Manifest.permission.CAMERA},
        REQUEST_CODE_CAMERA);
}

// Manejar la respuesta del usuario
@Override
public void onRequestPermissionsResult(int requestCode,
    String[] permissions, int[] grantResults) {
    if (requestCode == REQUEST_CODE_CAMERA) {
        if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            abrirCamara();
        } else {
```

```
        mostrarMensaje("Permiso de cámara denegado");  
    }  
}  
}
```

. CONCLUSIONES

A lo largo de esta monografía se han revisado los principales pilares del desarrollo de aplicaciones para la plataforma Android, desde la instalación del entorno de desarrollo hasta la publicación en Google Play Store. Este recorrido integral permite obtener una visión completa del ecosistema Android y de las competencias necesarias para construir aplicaciones de calidad profesional.

1. El entorno de desarrollo Android Studio, combinado con Gradle, proporciona un ecosistema robusto y eficiente que facilita la gestión de dependencias, la compilación y la depuración de aplicaciones, siendo indispensable dominar su configuración para trabajar de manera productiva.
 2. Comprender la arquitectura de Android y sus componentes fundamentales (Activity, Fragment, Service, BroadcastReceiver) es crucial para diseñar aplicaciones escalables, mantenibles y que aprovechen correctamente las capacidades del sistema operativo.
 3. Java, como lenguaje de programación base, ofrece toda la capacidad necesaria para implementar la lógica de negocio de las aplicaciones. Su manejo de tipos, estructuras de control orientadas a objetos y el paradigma de programación orientada a objetos son fundamentos sólidos que permiten escribir código limpio y reutilizable.
 4. El diseño de interfaces mediante XML y los principios de Material Design permite crear aplicaciones visualmente atractivas, coherentes y con una excelente experiencia de usuario, siguiendo los estándares establecidos por Google.
 5. El ciclo de vida de una Activity es uno de los conceptos más críticos en Android. Ignorarlo puede ocasionar pérdida de datos, consumo excesivo de batería y fallos inesperados en la aplicación. Su correcta gestión es sinónimo de aplicaciones estables y confiables.
 6. La gestión de datos, tanto local (SQLite, Room, SharedPreferences) como remota (APIs REST con Retrofit), representa el núcleo funcional de la mayoría de las aplicaciones modernas. La
-

arquitectura MVVM con LiveData y ViewModel garantiza una separación de responsabilidades que facilita el mantenimiento y las pruebas del código.

7. Finalmente, el proceso de publicación en Google Play requiere atención a los aspectos de seguridad, permisos y firma de la aplicación, aspectos que van más allá de la programación pura y que forman parte de la responsabilidad profesional del desarrollador de software.

En síntesis, el desarrollo de aplicaciones Android es una disciplina amplia y en constante evolución que combina habilidades técnicas de programación, diseño de interfaces, arquitectura de software y conocimiento del ecosistema móvil. Los conceptos aprendidos en este curso constituyen una base sólida sobre la cual continuar construyendo experiencia y conocimiento especializado.

. BIBLIOGRAFÍA

1. Android Developers. (2024). Android Studio User Guide. Google LLC.
<https://developer.android.com/studio>
2. Android Developers. (2024). Guide to app architecture. Google LLC.
<https://developer.android.com/topic/architecture>
3. Android Developers. (2024). Activity lifecycle. Google LLC.
<https://developer.android.com/guide/components/activities/activity-lifecycle>
4. Android Developers. (2024). Room Persistence Library. Google LLC.
<https://developer.android.com/training/data-storage/room>
5. Android Developers. (2024). LiveData Overview. Google LLC.
<https://developer.android.com/topic/libraries/architecture/livedata>
6. Android Developers. (2024). Retrofit Integration with Android. Google LLC.
<https://developer.android.com/training/basics/network-ops>
7. Blundell, P., & Bailey, D. (2021). Android Development Patterns: Best Practices for Professional Developers. Addison-Wesley Professional.
8. Google Material Design. (2024). Material Design 3. Google LLC. <https://m3.material.io>
9. Google Play. (2024). Play Console Help — Publish your app. Google LLC.
<https://support.google.com/googleplay/android-developer>
10. Griffiths, D., & Griffiths, D. (2021). Head First Android Development (3rd ed.). O'Reilly Media.
11. Karim, J. (2023). Android Programming: The Big Nerd Ranch Guide (5th ed.). Big Nerd Ranch Guides.
12. Square Inc. (2024). Retrofit – A type-safe HTTP client for Android and Java.
<https://square.github.io/retrofit/>
13. Square Inc. (2024). OkHttp – An HTTP client for Android. <https://square.github.io/okhttp/>
14. Square Inc. (2024). Moshi – A modern JSON library for Android.
<https://github.com/square/moshi>